Universidade do Minho

Ano Letivo: 2025/26

Mineração de Dados

Text Mining & NLP

Miguel Rocha | mrocha@di.uminho.pt
Diana Ferreira | diana@di.uminho.pt

Contents

1 Text Mining and NLP

2 Model Evolution

3 Key Concepts

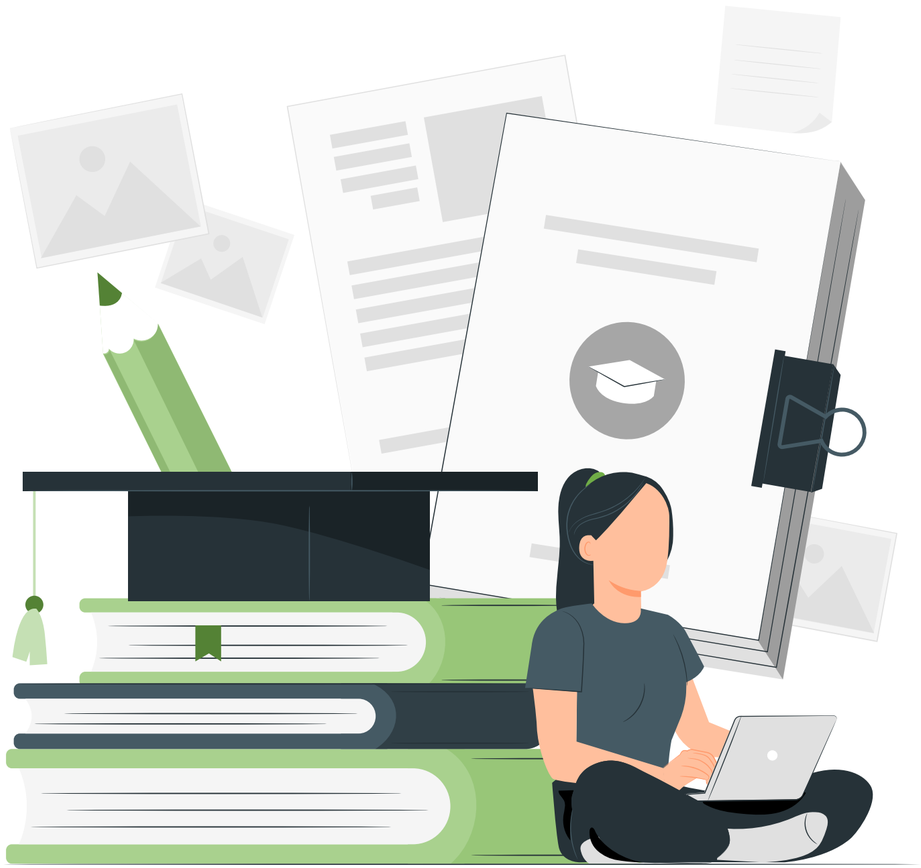
4 Practical Examples

Learning Objectives

By the end of this lecture, students should be able to:

- Know what Text Mining is and understand the typical pipeline.
- Understand the theoretical foundations of NLP.
- Explore the evolution from rule-based systems to modern Transformers.
- Master key techniques: tokenization, embeddings, attention mechanisms.
- Learn to implement state-of-the-art models using Hugging Face.
- Apply knowledge through hands-on exercises.

Bibliography



Books:

1. Vajjala, S., Majumder, B., Gupta, A., & Surana, H. (2020). Practical Natural Language Processing. O'Reilly
2. Geetha, T.V. (2024). Understanding Natural Language Processing – Machine Learning and Deep Learning Perspectives. Pearson.

Official Documentation:

1. NLTK Documentation
2. NLTK Book

Community and Resources:

1. **GitHub:**
   NLP-Data-Repository https://github.com/salzzyy/NLP-Data-Repository
   Truncation and Lemmatization Lab
   https://github.com/KPlanisphere/truncation-and-lemmatization

What is Text Mining?

Text Mining is the process of deriving high-quality information and patterns from unstructured text data.

Key Objectives:

- Extract previously unknown, understandable, and actionable patterns
- Transform unstructured text into structured data for analysis
- Discover knowledge that humans might miss

What does it involve?

- Text collection (e.g., scraping, APIs)
- Cleaning and preprocessing
- Feature extraction
- Analysis (statistics, ML, visualization)

What is Natural Language Processing (NLP)?

NLP is a discipline at the intersection of computer science, artificial intelligence (AI), and linguistics, aiming to equip machines with the ability to **understand, process,** and **generate human language**.

"I saw a man on a hill with a telescope."

These are some interpretations of the sentence shown above.

A. There is a man on the hill, and I watched him with my telescope.
B. There is a man on the hill, and he has a telescope.
C. I'm on a hill, and I saw a man using my telescope.
D. I'm on a hill, and I saw a man who has a telescope.

Core Challenge:
Language is inherently ambiguous, grammatically complex, and context-dependent.

Text Mining vs. NLP

Text Mining and NLP are both fields within the broader domain of computational linguistics, but they serve distinct purposes:

Criterion	Text Mining	NLP
Nature	Process/approach	Scientific/technological field
Objective	Extract useful information from text	Enable machines to understand language
Focus	Analytical application	Linguistic methods and algorithms
Scope	Includes NLP + statistics + ML	Part of AI and computational linguistics
Example	Analyze thousands of news articles	Identify entities in a text
Typical question	“What patterns exist in this corpus?”	“What does this sentence mean?”

The Intersection: NLP provides the tools; Text Mining provides the purpose.

Text Mining

uses → NLP

As humans, it can be difficult for us to understand the need for NLP, because our brains do it automatically (we understand the meaning, sentiment, and structure of text without processing it). But because computers are not humans, they need NLP to make sense of things.

Natural Language Processing (NLP)

➡ Subfields of NLP: NLU and NLG

Natural Language Understanding (NLU)

Focuses on a machine's ability to understand and interpret human language. It goes beyond basic word recognition, encompassing the understanding of intentions, sentiments, and complex contexts expressed in natural language.

PRACTICAL APPLICATIONS:
- Sentiment Analysis
- Intent Understanding in Chatbots

Natural Language Generation (NLG)

Focuses on generating natural language text from structured data. It involves transforming information into understandable narratives, producing texts that flow naturally as if written by humans.

PRACTICAL APPLICATIONS:
- Automated Reports
- Personalized Content Creation

In many AI systems, NLU and NLG work together to provide complete experiences, such as in virtual assistants that understand queries (NLU) and generate coherent responses (NLG).

Natural Language Processing (NLP)

➡ Applications of NLP

- **Automatic Language Translation**

DeepL, Google Translate

- **Virtual Assistants**

Siri, Alexa, and Google Assistant

- **Sentiment Analysis**
- **Information Retrieval**
- **Spam Detection**
- **Autocomplete**
- **Spell Checking**
- **Summarization of Text**
- **Speech Recognition**

Natural Language Processing (NLP)

Challenges

Ambiguity:

"I saw her duck"

duck = animal or action?

Synonyms:

"The car was parked outside the building"

car = vehicle = automobile → Different words but same meaning

Context:

"Apple released a product"

Apple = company, not fruit

Sarcasm:

"Great, my computer crashed again."

"Great" expresses frustration, not positive sentiment → Literal meaning ≠ intended meaning

NLP Evolution

Historical Perspective

1950s ——▶ 2026

Rule-Based	Statistical	Neural	Transformers
Chomsky	TF-IDF	Word2Vec	BERT
ELIZA	HMMs	RNN/LSTM	GPT
SHRDLU	MaxEnt	Attention	T5

Key Milestones:

- 1954: Georgetown-IBM translation experiment
- 1966: ELIZA chatbot
- 1990s: Statistical revolution
- 2013: Word2Vec
- 2017: Transformer architecture
- 2018: BERT, GPT
- 2020+: Large Language Models

Levels of NLP Analysis

➡ Lexical Analysis (Word Level)

Definition:
Breaks text into smaller units such as paragraphs, sentences, and words (tokens).

Focus:
- Word structure
- Vocabulary
- Tokenization

Example:
"The CEO of Microsoft spoke."
↓
"The", "CEO", "of", "Microsoft", "spoke"

Levels of NLP Analysis

➡ Syntactic Analysis (Grammar Level)

Definition:
Analyzes grammatical structure and relationships between words.

Focus:
- Sentence structure
- Word order
- Grammar validation

Example:
✓ "The shop goes to the house." (correct syntax)
✗ "Shop the house goes to." (incorrect syntax)

Levels of NLP Analysis

➡ Semantic Analysis (Meaning Level)

Definition:
Determines the meaning of words and sentences.

Focus:
- Literal meaning
- Logical consistency

Example:
✓ "The ice cream is cold."
✗ "The ice cream is hot." (semantically incorrect in normal context)

Levels of NLP Analysis

➡ Discourse Integration (Context Level)

Definition:
Analyzes meaning across multiple sentences using context.

Focus:
- Reference resolution
- Context understanding

Example:
"John works at Google. He is an engineer."
"He" → refers to John

Levels of NLP Analysis

➡ Pragmatic Analysis (Intent Level)

Definition:
Interprets language based on real-world context and speaker intent.

Focus:
- Intended meaning
- Situation context

Example:
"Can you open the window?"
Literal meaning: question about ability
Actual meaning: request to open the window

NLP Pipeline

The **NLP Pipeline** is a sequence of steps used to transform raw text into structured information that machines can analyze.

Raw Text

↓

Text Cleaning

Remove irrelevant characters, HTML, punctuation
Example: remove tags from scraped web pages

↓

Tokenization

Split text into smaller units (tokens)

↓

Stopword Removal

Remove common words without significant meaning, such as: the, is, in, at, of

↓

Stemming / Lemmatization

Reduce words to their base form
Example: running → run; better → good

↓

Feature Extraction

Convert text into numerical representation
Example:
Word frequency
TF-IDF
Embeddings

↓

NLP Tasks (NER, Classification, etc.)

Extract useful information:
- Named Entity Recognition (NER)
- Text classification
- Sentiment analysis
- Relationship extraction

↓

Structured Output / Knowledge

Key Principle: Garbage in, garbage out – preprocessing is critical!

Preprocessing – The "Dirty Work"

➡ Text Cleaning and Normalization

1. Character Normalization

1. Unicode normalization (é → e, café → cafe)
2. Normalize special characters
3. Handle emojis/emoticons

```
import unicodedata

text = "Café naïve façade"

normalized = unicodedata.normalize("NFKD", text)\
                         .encode("ascii", "ignore")\
                         .decode("utf-8")

print(normalized)
```

OUTPUT: Cafe naive facade

Preprocessing – The "Dirty Work"

➡ Text Cleaning and Normalization

2. Case Normalization

Converts text to lowercase (or consistent casing).

"Olá Mundo!" → "olá mundo!"

"NLP" → "nlp" *# But careful: "US" vs "us"*

```
text = "Microsoft AND Google Are Competitors"

normalized = text.lower()

print(normalized)
```

OUTPUT: microsoft and google are competitors

Preprocessing – The "Dirty Work"

➡ Text Cleaning and Normalization

3. Noise Removal

Removes:

- Punctuation
- Numbers (optional)
- URLs
- HTML tags
- Special characters
- Extra whitespace

```python
import re

text = "Visit https://example.com!!! This product costs $99.99 <br>"

# Remove URLs
text = re.sub(r"http\S+", "", text)

# Remove HTML tags
text = re.sub(r"<.*?>", "", text)

# Remove punctuation
text = re.sub(r"[^\w\s]", "", text)

print(text)
```

OUTPUT: Visit This product costs 9999

⚠ **Clean text → Better features → Better NLP performance**

Preprocessing – The "Dirty Work"

Python's Native Limitations

Native Python works at the **character and string level**, not at the **linguistic level**.

Limited Linguistic Awareness Python does not understand meaning, grammar, or context.

```
text = "Apple is looking at buying a startup"

print(text.lower().split())  ⟶  Output: ['apple', 'is', 'looking', 'at', 'buying', 'a', 'startup']
```

- **Python cannot identify "Apple" as an organization**
- **Cannot perform Named Entity Recognition (NER)**

Basic Tokenization Only Python splits text only using simple rules like whitespace.

```
text = "Hello, world!"

print(text.split())  ⟶  Output: ['Hello,', 'world!']
```

- **Punctuation remains attached to words**
- **Not linguistically correct tokenization**

Preprocessing – The "Dirty Work"

Python's Native Limitations

Native Python works at the **character and string level**, not at the **linguistic level**.

No Built-in Lemmatization or Stemming Python cannot reduce words to base forms.

```
words = ["running", "ran", "runs"]

# No native way to convert to "run"
```

- **Python cannot normalize word variations**

Manual and Inefficient Cleaning Python requires regex.

```
import re
text = "Hello!!! 123"
clean = re.sub(r"[^\w\s]", "", text)
print(clean)
```

→ `Output: ['Hello 123']`

- **Requires complex manual rules**
- **Error-prone and difficult to scale**

Libraries like **spaCy** and **NLTK** provide:
- Advanced tokenization
- Lemmatization
- Named Entity Recognition
- Efficient processing

Preprocessing – The "Dirty Work"

Tokenization: The Atomic Unit

What is Tokenization? Breaking down text into meaningful smaller units called tokens, such as words, phrases, or sentences.

Types of Tokenization

Word Tokenization:
Input: "I love NLP!"
Output: ["I", "love", "NLP", "!"]

Punctuation marks are usually treated as separate tokens.

Subword Tokenization (BERT):
Input: "unhappiness"
Output: ["un", "happiness"]

Character Tokenization:
Input: "love"
Output: ["l", "o", "v", "e"]

Sentence Tokenization:
Input: "I love programming. It's my passion."
Output: ["I love programming.", "It's my passion."]

Preprocessing – The "Dirty Work"

➡ Common Tokenization Techniques

Whitespace Tokenization

The simplest form of tokenization, where tokens are separated based on whitespace characters (spaces, tabs, newlines). This is the default tokenization in many programming languages.

Input: ”I love coding”
Output: ["I", "love", "coding"]

Strengths:
- Simple and fast
- Easy to implement
- Works reasonably well for basic tasks

Limitations:
- Cannot handle punctuation properly
- Cannot handle compound words
- Cannot handle subwords or unknown words

Use Cases:
- Basic NLP tasks
- Simple preprocessing
- Traditional text mining

Preprocessing – The "Dirty Work"

➡ Common Tokenization Techniques

Regular Expressions (Regex) Tokenization

Uses regex patterns to split the text.

Input: "Microsoft, founded in 1975, is a tech company."
Pattern: \w+

Output: ["Microsoft", "founded", "in", "1975", "is", "a", "tech", "company"]

Removes punctuation using a pattern that matches words only

Strengths:
- More flexible than whitespace tokenization
- Handles punctuation properly
- Customizable

Limitations:
- Requires manual rule definition
- Difficult to handle all edge cases
- Language-dependent

Use Cases:
- Controlled preprocessing
- Rule-based NLP systems

Preprocessing – The "Dirty Work"

➡ Common Tokenization Techniques

Byte Pair Encoding (BPE)

BPE is a subword tokenization technique that starts by treating the entire text as a sequence of characters. The most frequent pair of characters is merged into a single token, and this process is repeated iteratively.

Input: ["low", "lower", "newest", "widest"]
Output: ["low", “low”, “er”, “new”, “est”, “wid”, “est” "]

Step 1: Initial vocabulary = {l, o, w, e, r, n, s, t, i, d}
Step 2: Count adjacent pairs: ("e","s") appears 2x → merge to "es"
Step 3: New vocabulary: {l, o, w, e, r, n, s, t, es}
Step 4: Count remaining pairs: ("es","t") appears 2x → merge to "est"

Strengths:
- Handles rare and Out-of-Vocabulary (OOV) words better
- Reduces vocabulary size

Limitations:
- May split words unnaturally
- Requires training on corpus

Use Cases:
- GPT models
- Neural Machine Translation

Preprocessing – The "Dirty Work"

➡ Common Tokenization Techniques

WordPiece Tokenization

Similar to BPE but uses a probabilistic model to merge the most frequent pairs of subwords into one token. WordPiece was popularized by Google's BERT model.

Input: playing
Output: ["play", "##ing"]

⚠ "##" indicates continuation of a word

Strengths:
- Handles unknown words well
- Efficient vocabulary
- Better linguistic consistency than BPE

Limitations:
- More complex
- Requires training

Use Cases:
- BERT
- Modern NLP systems

Preprocessing – The "Dirty Work"

➡ Common Tokenization Techniques

SentencePiece Tokenization

An unsupervised text tokenizer and detokenizer that treats text as a sequence of characters. SentencePiece builds a vocabulary from the corpus, often used in machine translation systems.

Input: "New York"
Output: ["_New", "_York"]

⚠ "_" represents space

Strengths:
- Language independent
- Works well for languages without spaces (Chinese, Japanese)
- Fully data-driven

Limitations:
- Less intuitive
- Requires training

Use Cases:
- T5
- LLaMA
- Modern LLMs

Preprocessing – The "Dirty Work"

➡ Sentence Tokenization

```
from nltk.tokenize import sent_tokenize

text = "Hello everyone. Welcome to the class. You are studying NLP."
sent_tokenize(text)
```

Output:

['Hello everyone.',
'Welcome to the class.',
'You are studying NLP.']

```
import nltk.data

# Loading PunktSentenceTokenizer using English pickle file
tokenizer = nltk.data.load('tokenizers/punkt/PY3/english.pickle')
tokenizer.tokenize(text)
```

Output:

['Hello everyone.',
'Welcome to the class.',
'You are studying NLP.']

Preprocessing – The "Dirty Work"

➡ Word Tokenization

```
from nltk.tokenize import word_tokenize

text = "Hello everyone. Welcome to the class."
word_tokenize(text)
```

Output:

['Hello', 'everyone', '.', 'Welcome', 'to', 'the', 'class', '.']

```
from nltk.tokenize import TreebankWordTokenizer

tokenizer = TreebankWordTokenizer()
tokenizer.tokenize(text)
```

Output:

['Hello', 'everyone', '.', 'Welcome', 'to', 'the', 'class', '.']

Word Tokenization using Regular Expression

```
from nltk.tokenize import RegexpTokenizer

tokenizer = RegexpTokenizer(r'\w+')
text = "Let's see how it's working."
tokenizer.tokenize(text)
```

Output:

['Let', 's', 'see', 'how', 'it', 's', 'working']

Preprocessing – The "Dirty Work"

➡ Stopwords

Common words that often carry little meaning.

Traditional View: Remove "the", "is", "at" to save space and focus on "keywords."

Modern perspective: Stopword removal is often harmful for Transformers and LLMs because these models rely on function words to understand relationship and context.

💡 **Rule of Thumb:** Remove for simple TF-IDF classification; keep for anything "Deep."

✅ **Remove for:**
- Topic modeling
- Document clustering
- Keyword extraction

❌ **Keep for:**
- Sentiment analysis ("not good" vs "good")
- Style analysis
- Authorship attribution
- Translation

Preprocessing – The "Dirty Work"

Stemming

What is Stemming? Removing suffixes or prefixes from words to obtain their root form.

- Stemming generates the base word from the given word by removing the affixes of the word.
- It has a set of pre-defined rules that guide the dropping of these affixes.
- It must be noted that stemmers might not always result in semantically meaningful base words.

```python
from nltk.stem import PorterStemmer

# create an object of class PorterStemmer
porter = PorterStemmer()
print(porter.stem("Communication"))
```

Output:

Commun

```python
from nltk.stem import PorterStemmer

# create an object of class PorterStemmer
porter = PorterStemmer()
print(porter.stem("play"))
print(porter.stem("playing"))
print(porter.stem("plays"))
print(porter.stem("played"))
```

Output:

play
play
play
play

The stemmer reduces the word 'communication' to a base word 'commun' which is meaningless in itself.

Preprocessing – The "Dirty Work"

➡ Lemmatization

What is Lemmatization? Reducing words to their base or dictionary form while still ensuring they are valid words.

- Lemmatization involves grouping together the inflected forms of the same word.
- This way, we can reach out to the base form of any word which will be meaningful in nature.
- The base from here is called the Lemma.

```python
from nltk.stem import WordNetLemmatizer

# create an object of class WordNetLemmatizer
lemmatizer = WordNetLemmatizer()
print(lemmatizer.lemmatize("Communication"))
```

Output:

Communication

```python
from nltk.stem import WordNetLemmatizer

# create an object of class WordNetLemmatizer
lemmatizer = WordNetLemmatizer()
print(lemmatizer.lemmatize("plays", 'v'))
print(lemmatizer.lemmatize("played", 'v'))
print(lemmatizer.lemmatize("play", 'v'))
print(lemmatizer.lemmatize("playing", 'v'))
```

Output:

play
play
play
play

Preprocessing – The "Dirty Work"

➡ Stemming vs. Lemmatization

Feature	Stemming	Lemmatization
Logic	Chop word endings	Morphological analysis (dictionary-based)
Method	Rule-based heuristics	Vocabulary + morphology
Output	May not be real word	Always real word
Accuracy	Lower (over-stemming)	Higher (context-aware)
Speed	Fast	Moderate (requires POS tagging)
Example	“studies” → ”studi”	“studies” → ”study”

Preprocessing – The "Dirty Work"

➡ POS(Parts-Of-Speech) Tagging

PoS tagging is a fundamental task in NLP where each word in a sentence is assigned a grammatical category such as noun, verb, adjective or adverb.

1. Parts of Speech: Categories like nouns, verbs, adjectives, etc that define the role of a word in a sentence.

2. Tagging: The process of assigning a specific part-of-speech label to each word in a sentence.

EXAMPLE:

"**The quick brown fox jumps over the lazy dog.**"

After performing POS Tagging:

- *"The" is tagged as determiner (DT)*
- *"quick" is tagged as adjective (JJ)*
- *"brown" is tagged as adjective (JJ)*
- *"fox" is tagged as noun (NN)*
- *"jumps" is tagged as verb (VBZ)*
- *"over" is tagged as preposition (IN)*
- *the" is tagged as determiner (DT)*
- *"lazy" is tagged as adjective (JJ)*
- *"dog" is tagged as noun (NN)*

Preprocessing – The "Dirty Work"

➡ POS(Parts-Of-Speech) Tagging

```python
import nltk
from nltk.tokenize import word_tokenize
from nltk import pos_tag
nltk.download('punkt_tab’) #download the necessary datasets
nltk.download('averaged_perceptron_tagger_eng’)

text = "NLTK is a powerful library for natural language processing."
words = word_tokenize(text)

pos_tags = pos_tag(words)

print("Original Text:")
print(text)

print("\nPoS Tagging Result:")
for word, pos_tag in pos_tags:
    print(f"{word}: {pos_tag}")
```

```
Original Text:
NLTK is a powerful library for natural language processing.

PoS Tagging Result:
NLTK: NNP
is: VBZ
a: DT
powerful: JJ
library: NN
for: IN
natural: JJ
language: NN
processing: NN
.: .
```

Preprocessing – The "Dirty Work"

➡ Named Entity Recognition (NER)

NER involves identifying and classifying **entities** in text into predefined categories such as people, organizations, locations, dates, and more.

Key Categories of Entities:
- **Person Names:** e.g., Albert Einstein
- **Organizations:** e.g., OpenAI
- **Locations:** e.g., Paris
- **Dates/Times:** e.g., “23 February 2026”
- **Miscellaneous:** e.g., currencies, percentages, products

Example:

Text: “Imagine Dragons gave a concert in Lisbon, Portugal in 2025.”

NER Output:
Person → Imagine Dragons
Location → Portugal
Date → 2025

Preprocessing – The "Dirty Work"

➡ Named Entity Recognition (NER)

```python
from nltk import word_tokenize, pos_tag, ne_chunk

# Download the required resource for NER
nltk.download('maxent_ne_chunker_tab')
nltk.download('words') # This resource is also needed for the chunker

# Sample text
text = "Barack Obama was born in Hawaii in 1961."

# Tokenize and POS tag the sentence
tokens = word_tokenize(text)
tags = pos_tag(tokens)

# Apply Named Entity Recognition
entities = ne_chunk(tags)
print(entities)
```

```
[nltk_data] Downloading package maxent_ne_chunker_tab to
[nltk_data]     /root/nltk_data...
[nltk_data]   Unzipping chunkers/maxent_ne_chunker_tab.zip.
[nltk_data] Downloading package words to /root/nltk_data...
[nltk_data]   Unzipping corpora/words.zip.
(S
  (PERSON Barack/NNP)
  (PERSON Obama/NNP)
  was/VBD
  born/VBN
  in/IN
  (GPE Hawaii/NNP)
  in/IN
  1961/CD
  ./.)
```

Feature Extraction

➡ Bag-of-Words (BoW) Representation

Bag-of-Words (BoW) is a method to convert text into numerical vectors based on word frequency.

→ It represents text by counting how many times each word appears, ignoring grammar and word order.

Step 1: Example Text

Sentence 1:
"I love NLP"

Sentence 2:
"I love Python"

Step 2: Create Vocabulary

Unique words:
[I, love, NLP, Python]

Step 3: Count Word Frequencies

Sentence	I	love	NLP	Python
Sentence 1	1	1	1	0
Sentence 2	1	1	0	1

Feature Extraction

Bag-of-Words (BoW) Representation

```python
from sklearn.feature_extraction.text import CountVectorizer

corpus = [
    "I love NLP",
    "I love Python"
]

vectorizer = CountVectorizer()
X = vectorizer.fit_transform(corpus)

print("\nUsando scikit-learn:")
print("Matriz esparsa:\n", X.toarray())
print("Features:", vectorizer.get_feature_names_out())
```

Advantages

- Simple and easy to implement
- Works well for basic NLP tasks
- Efficient and fast

Limitations

- Loses word order
- Ignores context and meaning
- Cannot capture semantic relationships

Feature Extraction

➡ TF-IDF (Term Frequency-Inverse Document Frequency)

TF-IDF is a statistical measure to evaluate how important a word is to a document in a collection.

- **TF:** How **often** a term appears in a document.

$$TF(t,d) = \frac{\text{Number of times term } t \text{ appears in document } d}{\text{Total number of terms in document } d}$$

- **IDF:** Measures how **rare** a term is across all documents.

$$IDF(t) = \log \frac{\text{Total number of documents}}{\text{Number of documents containing term } t}$$

- **TF-IDF Score**: $TF\text{-}IDF(t,d) = TF(t,d) \times IDF(t)$

- High TF-IDF → term is important to the document and rare in the corpus.
- Low TF-IDF → term is common across documents (e.g., “the”, “and”).

Feature Extraction

➡ TF-IDF (Term Frequency-Inverse Document Frequency)

Corpus (3 documents):
1. Doc1: “the cat sat on the mat”
2. Doc2: “the dog sat on the log”
3. Doc3: “the cat chased the dog”

Step 1: Compute TF

Term	Doc1	Doc2	Doc3
the	2/6	2/6	2/5
cat	1/6	0	1/5
dog	0	1/6	1/5
sat	1/6	1/6	0
mat	1/6	0	0
log	0	1/6	0
chased	0	0	1/5

Step 2: Compute IDF

Example:
- “cat” → appears in 2 docs → IDF = log(3/2) ≈ 0.176
- “mat” → appears in 1 doc → IDF = log(3/1) ≈ 1.098

Step 3: Compute TF-IDF

High TF-IDF → “mat” in Doc1 (rare and relevant)
Low TF-IDF → “the” (common across docs)

Feature Extraction

TF-IDF

```python
from sklearn.feature_extraction.text import TfidfVectorizer

# Sample corpus
documents = [
"the cat sat on the mat",
"the dog sat on the log",
"the cat chased the dog"
]

# Initialize TF-IDF Vectorizer
vectorizer = TfidfVectorizer()
tfidf_matrix = vectorizer.fit_transform(documents)

# Convert to DataFrame for readability
import pandas as pd
df = pd.DataFrame(tfidf_matrix.toarray(),
columns=vectorizer.get_feature_names_out())
print(df)
```

	cat	chased	dog	log	mat	sat	the
Doc1	0.35	0	0	0	0.52	0.35	0.35
Doc2	0	0	0.35	0.52	0	0.35	0.35
Doc3	0.40	0.55	0.40	0	0	0	0.32

Advantages
- Simple and easy to implement
- Highlights important terms
- Language-agnostic

Limitations
- Loses word order
- Ignores context and meaning
- Cannot capture semantic relationships

- Rare words like **"mat"** or **"chased"** get higher TF-IDF → more important for document distinction.
- Common words like **"the"** are downweighted.

Feature Extraction

➡ N-grams and Context

- **N-grams** are contiguous sequences of **n items** (words or characters) from a text.
- They help capture **local context** and word patterns beyond single words.

Types of N-grams:

1.Unigram (n=1): Single words

Example: "the cat sat" → ["the", "cat", "sat"]

2.Bigram (n=2): Pairs of consecutive words

Example: "the cat sat" → ["the cat", "cat sat"]

3.Trigram (n=3): Triplets of consecutive words

Example: "the cat sat" → ["the cat sat"]

- n=1: Simple, sparse context
- n=2: Local context, still sparse
- n=3: Richer, very sparse
- n>3: Usually too sparse to be useful

- Bigrams/trigrams provide **more context.**
- Tradeoff: Higher n → more context but **sparser data**.

Feature Extraction

N-grams and Context

```python
from sklearn.feature_extraction.text import CountVectorizer

corpus = ["the cat sat on the mat", "the dog sat on the log"]
vectorizer = CountVectorizer(ngram_range=(2,2))  # bigrams
X = vectorizer.fit_transform(corpus)

import pandas as pd
df = pd.DataFrame(X.toarray(),
columns=vectorizer.get_feature_names_out())
print(df)
```

Advantages
- Captures local context
- Simple and interpretable
- Flexible granularity

Limitations
- Data sparsity
- Memory-intensive
- Limited long-range context
- Ignores semantics

the cat	cat sat	sat on	on the	the mat	the dog	dog sat	the log
1	1	1	1	1	0	0	0
0	0	1	1	0	1	1	1

Analyzing similarities

➡ Vector Space Model (VSM)

- Represents **documents as vectors** in a multidimensional space.
- Each word in the vocabulary is a dimension.
- The value in that dimension is how many times the word appears (or TF-IDF).
- Allows **mathematical computation of document similarity**.

EXAMPLE:

Vocabulary: [cat, dog, sat, mat, log]

Document	cat	dog	sat	mat	log
Doc1: "cat sat on mat"	1	0	1	1	0
Doc2: "dog sat on log"	0	1	1	0	1

Now, Doc1 and Doc2 have become vectors:

Doc1 → [1, 0, 1, 1, 0]
Doc2 → [0, 1, 1, 0, 1]

- Each vector represents counts or TF-IDF weights of terms: ["cat", "dog", "sat", "on", "mat", "log"].
- Each document is a point in this 5Dspace.

Analyzing similarities

➡ Cosine Similarity

- We want to measure how similar two documents are.
- Cosine similarity looks at the angle between the vectors, not their size.
- Measures **similarity between two document vectors** using the cosine of the angle between them:

$$\text{Cosine Similarity} = \cos(\theta) = \frac{A \cdot B}{\|A\|\|B\|}$$

- If the angle is 0° → vectors point in the same direction → similarity = 1 (identical)
- If the angle is 90° → vectors are orthogonal → similarity = 0 (completely different)

EXAMPLE:

Doc1 [1,0,1,1,0]
Doc2 [0,1,1,0,1]

Document	cat	dog	sat	mat	log
Doc1: "cat sat on mat"	1	0	1	1	0
Doc2: "dog sat on log"	0	1	1	0	1

Only the word “sat” is common.. Cosine similarity ≈ 0.35 → they have some similarity, but not much.

Analyzing similarities

VSM and Cosine Similarity

```python
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

corpus = ["cat sat on mat", "dog sat on log"]
vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(corpus)

# Cosine similarity between all document pairs
cos_sim = cosine_similarity(X)
print(cos_sim)
```

Advantages

- Simple and intuitive
- Works well for document ranking
- Scalable

Limitations

- Data sparsity
- Ignores word order
- Vocabulary dependent
- Ignores semantics

```
OUTPUT:
[[1. 0.35]
[0.35 1. ]]
```

Diagonal (1.0) → each document is 100% similar to itself.
0.35 → similarity between Doc1 and Doc2.
They share some words (sat, on), but also have different words (cat vs dog, mat vs log).

NLP Libraries

➡ Popular NLP Libraries in Python

Python provides powerful libraries for Natural Language Processing tasks.

NLTK

spaCy

GENSIM
topic modelling for humans

fastText

TextBlob

Transformers

NLP Libraries

➡ NLTK (Natural Language Toolkit):

NLTK is a comprehensive library for teaching and research in NLP.

Features:
- Tokenization
- Stemming and Lemmatization
- Stopword removal
- POS tagging
- Access to linguistic datasets

Best for:
- Learning NLP concepts and basic text processing

Advantages:
- Excellent for learning and education
- Large number of built-in datasets
- Very well documented

Limitations:
- Slower than modern libraries
- Less efficient for large-scale applications

NLTK

NLP Libraries

➡ spaCy

spaCy is a modern, fast, and production-ready NLP library.

Features:
- Fast tokenization
- Named Entity Recognition (NER)
- POS tagging
- Dependency parsing
- Pretrained models

Best for:
- Production systems and real-world NLP applications

Advantages:
- Very fast and efficient
- Easy to use
- Designed for real-world applications

Limitations:
- Smaller number of built-in datasets than NLTK

spaCy

NLP Libraries

➡ GENSIM

Gensim is a library designed for topic modeling and document similarity analysis.

Features:
- Topic modeling (LDA)
- Word embeddings (Word2Vec)
- Document similarity
- Large-scale text processing

Best for:
- Topic modeling and semantic similarity analysis

Advantages:
- Efficient for large datasets
- Specialized in semantic analysis

Limitations:
- Not designed for basic NLP tasks like tokenization or NER

GENSIM
topic modelling for humans

NLP Libraries

➡ fastText

Library, developed by Meta AI, for efficient word embeddings and text classification.

Features:
- Word embeddings
- Text classification
- Very fast training
- Works well with small datasets

Best for:
- Word embeddings and fast text classification

Advantages:
- Very fast training and inference
- Handles unknown words using subword information
- Efficient memory usage

Limitations:
- Limited functionality compared to modern transformer models
- Limited contextual understanding

fastText

NLP Libraries

➡ TextBlob

TextBlob is a Python library designed for processing textual data.

Features:
- Part-of-Speech tagging
- Noun phrase extraction
- Sentiment analysis
- Classification
- Language translation
- Parsing

Best for:
- Sentiment analysis, basic NLP tasks, rapid prototyping, and educational purposes.

Advantages:
- Very easy to use
- Simple and beginner-friendly API
- Minimal setup required

Limitations:
- Limited functionality
- Lower accuracy than modern libraries
- Not suitable for large-scale applications

TextBlob

NLP Libraries

➡ Hugging Face Transformers

Provides access to pretrained state-of-the-art transformer models like BERT, GPT, and T5.

Features:
- Named Entity Recognition
- Text classification
- Question answering
- Text generation
- Sentiment analysis

Best for:
- Modern NLP and deep learning applications

Advantages:
- State-of-the-art performance
- Large number of pretrained models
- Supports many NLP tasks
- Strong community and documentation

Limitations:
- Requires more computational resources
- Slower than traditional NLP libraries
- Larger memory usage
- More complex for beginners

Transformers

Kahoot

Goal

- To **reward and compensate students** who actively attend and participate in classes
- To reinforce key concepts in an engaging and competitive way

Rules

- Only students **physically present in class** may participate
- No external help (notes, phones, AI tools, or collaboration)
- Any form of cheating results in **disqualification**

Scoring & Rewards

🥇 **1st place**: +0.5 bonus points

🥈 **2nd place**: +0.3 bonus points

🥉 **3rd place**: +0.2 bonus points

(Bonus points are added to the continuous assessment component.)

⚠️ Bonus points are **non-transferable** and **cannot exceed course limits**

Practical Application

➡ **Worksheet PL4**

Blackboard